

Reviewer Pack — Minimal Symplectic Laplacian Eigenvalues and Communication C...

Entry 7b4374687b90 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 20:26:39 UTC

Minimal Symplectic Laplacian Eigenvalues and Communication Complexity Rank

Entry ID: 7b4374687b90 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 20:26:39 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry (Laplacian Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For any given d -dimensional vector space V over a field F , the rank of its symplectic Laplacian matrix $L_S(V)$ is linearly correlated with the communication complexity rank $r(V)$, such that $\lambda_{\min}(L_S(V)) = \Theta(r(V))$, where $\lambda_{\min}(L_S(V))$ is the smallest non-zero eigenvalue of $L_S(V)$.

Rationale (proposer's reasoning):

Symplectic geometry, particularly the Laplacian operator, has been applied to graph theory and its connections with communication complexity. The symplectic Laplacian matrix provides a spectral characterization that might reveal structural properties of vector spaces relevant to communication complexity, potentially explaining the hardness of certain communication tasks.

Taxonomy category: symplectic_laplacian (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): beef398761eb12df

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of seeds yield a Pearson correlation coefficient between $\lambda_{\min}(L_S(V))$ and $r(V) \geq 0.7$, with no seed producing a p -value < 0.05 indicating non-significant correlation.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "symplectic Laplacian eigenvalues" AND "communication complexity rank"
- "Laplace matrix in symplectic geometry" AND "matrix rank in communication complexity" -
"eigenvalues of symplectic matrices" related to "rank in communication theory"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Define a function to generate a random d-dimensional vector space over a finite field F
    def generate_vector_space(d, q):
        V = []
        for _ in range(d):
            v = [random.randint(0, q-1) for _ in range(d)]
            V.append(v)
        return V

    # Define a function to compute the symplectic Laplacian matrix L_S(V)
    def symplectic_laplacian(V):
        d = len(V)
        L_S = [[0] * d for _ in range(d)]
        for i in range(d):
            for j in range(i+1, d):
                L_S[i][j] = V[i][j] - V[j][i]
                L_S[j][i] = -L_S[i][j]
        return L_S

    # Define a function to compute the eigenvalues of a matrix
    def eigenvalues(matrix):
        def determinant(A):
            if len(A) == 1:
                return A[0][0]
            det = Fraction(0, 1)
            for j in range(len(A)):
                submatrix = [row[:j] + row[j+1:] for row in A[1:]]
                det += (-1)**j * A[0][j] * determinant(submatrix)
```

```

        det += (-1)**j * A[0][j] * determinant(submatrix)
    return det

def characteristic_polynomial(matrix):
    n = len(matrix)
    poly = []
    for k in range(n, -1, -1):
        coeff = Fraction(0, 1)
        for i in range(k+1):
            coeff += (-1)**i * Fraction(determinant([row[:k] + row[k+1:] for row in matrix[:i+1]]))
        poly.append(coeff)
    return poly

def roots(poly):
    n = len(poly)
    if n == 1:
        return []
    elif n == 2:
        a, b = poly[0], poly[1]
        return [(-b + math.sqrt(b**2 - 4*a)) / (2*a), (-b - math.sqrt(b**2 - 4*a)) / (2*a)]
    else:
        # Use the Jenkins-Traub method for higher degree polynomials
        pass

poly = characteristic_polynomial(matrix)
return roots(poly)

# Define a function to compute the communication complexity rank r(V)
def communication_complexity_rank(V):
    d = len(V)
    r = 0
    for i in range(d):
        for j in range(i+1, d):
            if V[i][j] != 0:
                r += 1
    return r

# Generate a random d-dimensional vector space over a finite field F
d = random.randint(5, 40)
q = random.randint(2, 10)
V = generate_vector_space(d, q)

# Compute the symplectic Laplacian matrix L_S(V) and its eigenvalues
L_S = symplectic_laplacian(V)
eigenvals = eigenvalues(L_S)
lambda_min = min(eigenval for eigenval in eigenvals if eigenval != 0)

# Calculate the communication complexity rank r(V)
r_V = communication_complexity_rank(V)

# Correlate  $\lambda_{\min}(L_S(V))$  with  $r(V)$  using Pearson correlation
n = len(eigenvals)
x_mean = sum(eigenvals) / n
y_mean = r_V
x_var = sum((eigenval - x_mean)**2 for eigenval in eigenvals) / n
y_var = 0 # Since r(V) is a constant, its variance is zero

```

```

cov = sum((eigenval - x_mean) * (r_V - y_mean) for eigenval in eigenvals) / n
correlation = cov / math.sqrt(x_var * y_var)

# Check if the correlation is significant with a p-value threshold of 0.05
t_statistic = correlation * math.sqrt(n - 2) / math.sqrt(1 - correlation**2)
p_value = 2 * (1 - math.erf(abs(t_statistic) / math.sqrt(2)))

# Return the results as a dictionary
return {
    "metric_name": "Pearson Correlation",
    "metric_value": correlation,
    "instances_tested": n,
    "n_max": d,
    "conjecture_holds": p_value >= 0.05,
    "counterexample": "" if p_value >= 0.05 else f"p-value={p_value:.4f} < 0.05"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={support_fraction:.4f}")
    elif any(result["counterexample"] for result in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if result["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\"")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aa7b143f.py", line 126, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aa7b143f.py", line 91, in run_trial
    eigenvals = eigenvalues(L_S)
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aa7b143f.py", line 71, in eigenvalues
    poly = characteristic_polynomial(matrix)

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aa7b143f.py", line 56, in characteristic
    coeff += (-1)**i * Fraction(determinant([row[:k] + row[k+1:] for row in matrix if j != i]), math.factorial(k-i))
NameError: name 'j' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to an undefined variable 'j', preventing the production of data necessary to evaluate the conjecture. | next: Debug the test code to resolve the NameError and rerun the test with a valid seed.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15016
2	propose	ollama_remote	glm4:latest	0	0	16534
3	propose	ollama_remote	glm4:latest	0	0	16814
4	preregistration	ollama_remote	glm4:latest	0	0	9717
5	novelty	ollama_remote	glm4:latest	0	0	8567
6	novelty	ollama_remote	glm4:latest	0	0	12365
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21083
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14048
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16146
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15807
11	judge	ollama_remote	glm4:latest	0	0	8802

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 154899 ms total latency. Provider mix: {'ollama remote': 11}

(full prompt+response transcripts available in [research/audit/7b4374687b90.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7b4374687b90.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7b4374687b90.tar.gz (if generated)